



Plex Music Enhancer

Benutzerhandbuch

Version 1.1

Steinbecker Softwaredienst

MIT License

Copyright 2026 Steinbecker Softwaredienst

github.com/amylasehai0z/plex-music-enhancer

Erstellt am 9. Juli 2026

Inhaltsverzeichnis

1	Plex Music Enhancer	1
1.1	Benutzerhandbuch	1
1.2	Über dieses Handbuch	1
1.3	Navigationshinweise	1
1.4	Inhaltsverzeichnis	1
2	1. Einleitung	2
2.1	1.1 Welche Probleme löst das Programm?	2
2.2	1.2 Warum KI?	2
2.3	1.3 Nutzen	3
2.4	1.4 Grenzen	3
2.5	1.5 Sicherheitsphilosophie	3
2.6	1.6 Halluzinationsschutz	4
2.7	1.7 Begriffe: Preview, Review, Apply	4
2.8	1.8 Empfohlener Grundablauf	4
3	2. Installation	5
3.1	2.1 Voraussetzungen	5
3.2	2.2 Python installieren	5
	3.2.1 macOS	5
	3.2.2 Windows	5
	3.2.3 Linux	6
3.3	2.3 Git installieren	6
3.4	2.4 Repository klonen	6
3.5	2.5 Virtuelle Umgebung erstellen	6
3.6	2.6 Abhängigkeiten installieren	7
3.7	2.7 OpenAI API Key	7
3.8	2.8 Plex Token	7
3.9	2.9 Login	7
3.10	2.10 Doctor	8
3.11	2.11 Verifikation	8
3.12	2.12 Häufige Fehler	8
4	3. Erste Schritte	9
4.1	3.1 Vorbereitung	9
4.2	3.2 Doctor ausführen	9
4.3	3.3 Login ausführen	10
4.4	3.4 Vorschau erzeugen	10
4.5	3.5 Review starten	10

4.6	3.6 Entscheidung treffen	11
4.7	3.7 Apply ausführen	11
4.8	3.8 Ergebnis prüfen	11
4.9	3.9 Häufige Entscheidungen	11
5	4. Konfiguration	12
5.1	4.1 Konfigurationsdateien	12
5.2	4.2 Umgebungsvariablen	12
5.3	4.3 Plex-Konfiguration	13
5.4	4.4 Provider-Auswahl	13
5.5	4.5 Modell-Auswahl	13
5.6	4.6 Prompt-Versionen	14
5.7	4.7 Logging	14
5.8	4.8 Cache	14
5.9	4.9 Exportordner	15
5.10	4.10 Secrets	15
5.11	4.11 Best Practices	15
6	5. CLI-Befehle	16
6.1	5.1 Globale Optionen	16
6.2	5.2 version	16
6.3	5.3 doctor	16
6.4	5.4 serve	17
6.5	5.5 login	17
6.6	5.6 audit	17
6.7	5.7 plan	18
6.8	5.8 benchmark	18
6.9	5.9 capabilities	18
6.10	5.10 match	18
6.11	5.11 scan	19
6.12	5.12 inspect	19
6.13	5.13 probe write	19
6.14	5.14 metadata album	20
6.15	5.15 context album	20
6.16	5.16 preview	20
6.17	5.17 preview artist	20
6.18	5.18 review album	21
6.19	5.19 review artist	21
6.20	5.20 apply	22
6.21	5.21 apply artist	22
6.22	5.22 batch review	22
6.23	5.23 library	22
6.24	5.24 cache	22
6.25	5.25 Fehlerbehebung bei Befehlen	23
7	6. Workflows	24
7.1	6.1 Einzelnes Album	24
7.2	6.2 Einzelner Künstler	24

7.3	6.3 Ganze Bibliothek	25
7.4	6.4 Übersetzung	25
7.5	6.5 Verbesserung vorhandener deutscher Texte	25
7.6	6.6 Preview Workflow	25
7.7	6.7 Review Workflow	25
7.8	6.8 Apply Workflow	26
7.9	6.9 Batch Processing	26
7.10	6.10 Library Mode	26
7.11	6.11 Qualitätssicherung	26
7.12	6.12 Rollback-Strategie	26
7.13	6.13 Best Practices	27
8	7. KI und Editorial Engine	28
8.1	7.1 Gesamtarchitektur	28
8.2	7.2 Plex	29
8.3	7.3 Metadata Providers	29
8.4	7.4 MusicBrainz	29
8.5	7.5 Wikipedia	29
8.6	7.6 Discogs	29
8.7	7.7 Last.fm	29
8.8	7.8 Context Builder	30
8.9	7.9 Editorial Style Engine	30
8.10	7.10 GPT-5.5 oder anderes Modell	30
8.11	7.11 Prompt Engineering	30
8.12	7.12 Prompt-Versionen	30
8.13	7.13 Warum Texte factual bleiben	31
9	8. Review-System	32
9.1	8.1 Quality Score	32
9.2	8.2 Checks	32
9.3	8.3 Quality Summary	33
9.4	8.4 Language Detection	33
9.5	8.5 Length Validation	33
9.6	8.6 Editorial Validation	33
9.7	8.7 Apply-Policy	33
9.8	8.8 Diff Viewer	34
9.9	8.9 Approval Process	34
9.10	8.10 Manuelle Bearbeitung	34
9.11	8.11 Warum Summaries scheitern können	35
9.12	8.12 Typischer Review Workflow	35
9.13	8.13 Debug-Dateien	35
10	9. Provider	37
10.1	9.1 Plex	37
10.2	9.2 MusicBrainz	37
10.3	9.3 Wikipedia	37
10.4	9.4 Discogs	38
10.5	9.5 Last.fm	38

10.6	9.6 OpenAI	38
10.7	9.7 Fehlerbehandlung	38
11	10. Cache	39
11.1	10.1 Zweck	39
11.2	10.2 Speicherorte	39
11.3	10.3 Lebensdauer	39
11.4	10.4 Cache prüfen	39
11.5	10.5 Cache löschen	40
11.6	10.6 Cache neu aufbauen	40
11.7	10.7 Wann löschen?	40
11.8	10.8 Performance-Beispiele	40
12	11. Fehlersuche	41
12.1	11.1 Keine Verbindung zu Plex	41
12.2	11.2 Authentifizierung fehlgeschlagen	41
12.3	11.3 Album nicht gefunden	42
12.4	11.4 MusicBrainz nicht verfügbar	42
12.5	11.5 Wikipedia nicht verfügbar	42
12.6	11.6 OpenAI Timeout	42
12.7	11.7 Rate Limits	42
12.8	11.8 Providerfehler	43
12.9	11.9 Cache-Probleme	43
12.10	11.10 Review fehlgeschlagen	43
12.11	11.11 Apply fehlgeschlagen	43
12.12	11.12 Konfigurationsfehler	43
12.13	11.13 API Key fehlt	44
12.14	11.14 Netzwerkprobleme	44
13	12. Häufig gestellte Fragen	45
13.1	Grundlagen	45
13.1.1	1. Was macht Plex Music Enhancer?	45
13.1.2	2. Verändert das Programm meine Plex-Bibliothek automatisch?	45
13.1.3	3. Brauche ich Programmierkenntnisse?	45
13.1.4	4. Kann ich das Programm ohne OpenAI nutzen?	45
13.1.5	5. Warum gibt es einen DummyProvider?	46
13.1.6	6. Warum nutzt Preview nicht OpenAI, obwohl ein API Key gesetzt ist?	46
13.1.7	7. Welche Python-Version brauche ich?	46
13.1.8	8. Funktioniert das auf macOS, Windows und Linux?	46
13.1.9	9. Muss Plex lokal laufen?	46
13.1.10	10. Funktioniert das mit Plexamp?	46
13.2	Installation und Einrichtung	46
13.2.1	11. Wie prüfe ich, ob die Installation funktioniert?	46
13.2.2	12. Was macht <code>plex-enhancer login</code> ?	46
13.2.3	13. Wird mein Plex Token angezeigt?	47
13.2.4	14. Darf ich <code>.env</code> in Git speichern?	47
13.2.5	15. Woher bekomme ich mein Plex Token?	47
13.2.6	16. Was bedeutet "configuration missing"?	47

13.2.7	17. Warum schlägt doctor fehl?	47
13.2.8	18. Kann ich mehrere Plex Server nutzen?	47
13.2.9	19. Welche Datei ist wichtiger: .env oder Umgebungsvariablen?	47
13.2.10	20. Warum wird mein OpenAI Key nicht erkannt?	47
13.3	Plex und Bibliotheken	47
13.3.1	21. Welche Plex-Bibliotheken werden verarbeitet?	48
13.3.2	22. Wie finde ich heraus, welche Musikbibliotheken erkannt werden?	48
13.3.3	23. Wie exportiere ich Bibliotheksstatistiken?	48
13.3.4	24. Warum wird ein Album nicht gefunden?	48
13.3.5	25. Was ist ein Rating Key?	48
13.3.6	26. Kann ich nach ID statt nach Name suchen?	48
13.3.7	27. Werden Tracks verändert?	48
13.3.8	28. Kann Plex Music Enhancer Playlists bearbeiten?	48
13.3.9	29. Kann ich nur eine bestimmte Bibliothek planen?	48
13.3.10	30. Was passiert bei doppelten Albumtiteln?	48
13.4	KI und Texte	49
13.4.1	31. Warum erfindet GPT keine Chartpositionen?	49
13.4.2	32. Warum sind Produzenten manchmal nicht enthalten?	49
13.4.3	33. Warum sind die Texte konservativ?	49
13.4.4	34. Warum ist ein Text kürzer als erwartet?	49
13.4.5	35. Warum klingt ein Text manchmal allgemein?	49
13.4.6	36. Kann ich einen Text neu erzeugen?	49
13.4.7	37. Kann ich manuell bearbeiten?	49
13.4.8	38. Warum wird Markdown abgelehnt?	49
13.4.9	39. Warum werden Bullet-Listen abgelehnt?	49
13.4.10	40. Kann ich englische Texte übersetzen?	50
13.4.11	41. Kann ich deutsche Texte verbessern?	50
13.4.12	42. Wird beim Übersetzen zusammengefasst?	50
13.4.13	43. Was passiert, wenn der vorhandene Text schon gut ist?	50
13.4.14	44. Was bedeutet REVIEW im Plan?	50
13.4.15	45. Warum lehnt Review einen Text ab?	50
13.5	Provider	50
13.5.1	46. Was ist MusicBrainz?	50
13.5.2	47. Was ist eine MBID?	50
13.5.3	48. Was ist eine Release Group?	50
13.5.4	49. Was liefert Wikipedia?	51
13.5.5	50. Warum findet Wikipedia keinen Artikel?	51
13.5.6	51. Was liefert Discogs?	51
13.5.7	52. Was liefert Last.fm?	51
13.5.8	53. Sind Last.fm-Tags objektive Fakten?	51
13.5.9	54. Werden Providerfehler abgefangen?	51
13.5.10	55. Muss ich alle Provider konfigurieren?	51
13.6	Cache und Performance	51
13.6.1	56. Warum gibt es einen Cache?	51
13.6.2	57. Wo liegt der Cache?	51
13.6.3	58. Wie sehe ich Cache-Statistiken?	52
13.6.4	59. Wie liste ich Cache-Einträge?	52

13.6.5	60. Wie lösche ich den Cache?	52
13.6.6	61. Wann sollte ich den Cache löschen?	52
13.6.7	62. Wird ein fehlgeschlagener Provider dauerhaft gecacht?	52
13.6.8	63. Warum ist der erste Lauf langsam?	52
13.6.9	64. Warum ist ein großer Bibliothekslauf langsam?	52
13.6.10	65. Kann ich parallelisieren?	52
13.7	Review und Apply	52
13.7.1	66. Was ist der Unterschied zwischen Preview und Review?	52
13.7.2	67. Was ist der Unterschied zwischen Review und Apply?	53
13.7.3	68. Kann ich Änderungen rückgängig machen?	53
13.7.4	69. Wo liegen Audit-Dateien?	53
13.7.5	70. Was passiert, wenn Verifikation fehlschlägt?	53
13.7.6	71. Kann Apply den falschen Datensatz ändern?	53
13.7.7	72. Warum gibt es keine automatische Massenänderung ohne Review?	53
13.7.8	73. Kann ich Batch-Verarbeitung abbrechen?	53
13.7.9	74. Was macht Resume?	53
13.7.10	75. Wo sehe ich eine Zusammenfassung eines Bibliothekslaufs?	53
13.8	JSON, Exporte und Fehlersuche	54
13.8.1	76. Wo werden JSON-Dateien gespeichert?	54
13.8.2	77. Kann ich JSON-Ausgaben weiterverarbeiten?	54
13.8.3	78. Warum unterscheiden sich Rich-Ausgabe und JSON?	54
13.8.4	79. Wie finde ich heraus, welcher Prompt verwendet wurde?	54
13.8.5	80. Warum stimmen Prompt-Versionen im Text nicht mit alten Ausgaben überein?	54
13.8.6	81. Was mache ich bei unverständlichen Fehlern?	54
13.8.7	82. Wann soll ich <code>--verbose</code> nutzen?	54
13.8.8	83. Sind Stack Traces normal?	54
13.8.9	84. Kann ich die Dokumentation als PDF bauen?	54
13.8.10	85. Ist das Handbuch auch für gedruckte Nutzung gedacht?	55
14	13. Glossar	56
14.1	13.1 Empfohlene deutsche Begriffe	58
15	14. Anhang	60
15.1	14.1 Wichtige Befehle	60
15.2	14.2 Häufige Exportpfade	60
15.3	14.3 Konfigurationsbeispiele	61
15.4	14.4 Sicherer Einzelalbum-Ablauf	61
15.5	14.5 Sicherer Bibliotheksablauf	61
15.6	14.6 Exit-Code-Orientierung	61
15.7	14.7 Empfehlungen für produktive Nutzung	62
15.8	14.8 PDF-Erstellung	62
15.9	14.9 Backend-API und REST	62
15.10	14.10 Developer Mode	64
16	15. Container, Portainer und Synology	65
16.1	15.1 Docker Image	65
16.2	15.2 Docker Compose	66

16.3	15.3 Volumes	66
16.4	15.4 Umgebungsvariablen	67
16.5	15.5 Portainer	67
	16.5.1 Deployment als Stack	68
	16.5.2 Deployment als einzelner Container	68
	16.5.3 GHCR-Image	68
	16.5.4 Volumes	68
	16.5.5 Environment-Variablen	69
	16.5.6 Port-Mapping und Healthcheck	69
	16.5.7 Container aktualisieren	69
	16.5.8 Rollback, Logs und Neustart	69
16.6	15.6 Synology Container Manager	69
16.7	15.7 CI/CD	70
17	16. Release Management	71
17.1	16.1 Empfohlener Release-Prozess	71
17.2	16.2 GitHub Releases	72
17.3	16.3 Release-Artefakte	72
17.4	16.4 GHCR und Image Tags	72
17.5	16.5 Build Report	72
17.6	16.6 Docker Analyse	73
17.7	16.7 Release Readiness Report	73
17.8	16.8 Portainer nach dem Release	74
18	Anhang zur PDF-Ausgabe	75
18.1	Versionshistorie	75
18.2	CLI-Referenz	75
18.3	Lizenz	75

Tabellenverzeichnis

Abbildungsverzeichnis

Kapitel 1

Plex Music Enhancer

1.1 Benutzerhandbuch

Version 1.1

Copyright © 2026 Steinbecker Softwaredienst

Lizenz: MIT License

Projektseite: <https://github.com/amylasehai0z/plex-music-enhancer>

Erstellt am: 2026-07-09

1.2 Über dieses Handbuch

Dieses Handbuch beschreibt die Benutzung von Plex Music Enhancer v1.1. Es richtet sich an Anwenderinnen und Anwender, die ihre Plex-Musikbibliothek kontrolliert mit besseren deutschen Album- und Künstlertexten pflegen möchten.

Das Handbuch ist für GitHub, PDF-Export, MkDocs und gedruckte Ausgabe geeignet.

1.3 Navigationshinweise

- Befehle werden in Codeblöcken gezeigt.
- Wichtige Hinweise stehen in markierten Abschnitten.
- Kapitel verweisen mit relativen Links aufeinander.
- Technische Begriffe werden im [Glossar](#) erklärt.

1.4 Inhaltsverzeichnis

Beim PDF-Build wird das Inhaltsverzeichnis automatisch von Pandoc erzeugt.

Kapitel 2

1. Einleitung

Plex Music Enhancer ist ein Kommandozeilenwerkzeug für die Pflege von Musikmetadaten in Plex. Es liest bestehende Plex-Informationen, ergänzt sie mit externen Quellen und erzeugt daraus deutsche, sachliche und stilistisch konsistente Album- und Künstlertexte.

2.1 1.1 Welche Probleme löst das Programm?

Typische Musikbibliotheken enthalten:

- Alben ohne Beschreibung
- englische Zusammenfassungen
- maschinell klingende Übersetzungen
- uneinheitliche Schreibweise
- fehlende Quelleninformationen
- zu kurze oder werbliche Texte

Plex Music Enhancer löst diese Probleme nicht durch blindes Überschreiben, sondern durch einen kontrollierten Prozess aus Vorschau, Prüfung und sicherer Anwendung.

2.2 1.2 Warum KI?

Metadatenquellen liefern Fakten, aber selten einen flüssigen deutschen Text. Ein Sprachmodell kann aus strukturierten Fakten eine gut lesbare Beschreibung formulieren.

Wichtig: Die KI wird nicht als freie Suchmaschine verwendet. Sie erhält einen begrenzten Kontext und klare Regeln. Dadurch wird das Risiko erfundener Informationen reduziert.

2.3 1.3 Nutzen

Nutzen	Beschreibung
bessere Lesbarkeit	deutsche, flüssige Beschreibungen
mehr Konsistenz	einheitlicher Stil in der Bibliothek
Sicherheit	keine automatische Änderung ohne Freigabe
Nachvollziehbarkeit	JSON-Exporte, Backups und Auditdateien
Skalierbarkeit	Batch- und Library-Modus

2.4 1.4 Grenzen

Plex Music Enhancer kann fehlende Fakten nicht herbeizaubern. Wenn keine Quelle Informationen zu Produzenten, Charts oder Aufnahmestudios enthält, werden diese Informationen nicht erfunden.

Grenzen:

- keine Garantie auf vollständige öffentliche Metadaten
- KI-Ergebnisse müssen geprüft werden
- optionale Provider benötigen Zugangsdaten
- Netzwerkdienste können zeitweise nicht erreichbar sein

2.5 1.5 Sicherheitsphilosophie

```

flowchart TD
    A["Plex lesen"] --> B["Kontext sammeln"]
    B --> C["Text erzeugen"]
    C --> D["Qualität prüfen"]
    D --> E["Review anzeigen"]
    E --> F{"Benutzerentscheidung"}
    F -->|Apply| G["Backup erstellen"]
    G --> H["Nach Plex schreiben"]
    H --> I["Neu laden und prüfen"]
    F -->|Skip/Quit| J["Keine Änderung"]
  
```

Plex wird nur verändert, wenn Sie ausdrücklich Apply wählen oder den Apply-Befehl ausführen.

2.6 1.6 Halluzinationsschutz

Das Programm minimiert erfundene Aussagen durch:

- strukturierte Providerdaten
- MusicBrainz-Matching
- Fact Verification
- Prompt-Regeln
- Editorial Engine
- Quality Engine
- Review durch den Menschen

2.7 1.7 Begriffe: Preview, Review, Apply

Begriff	Bedeutung
Preview	Text erzeugen und ansehen, ohne Plex zu ändern
Review	Text, Diff und Qualitätsprüfung interaktiv prüfen
Apply	geprüften Text mit Backup und Verifikation nach Plex schreiben

2.8 1.8 Empfohlener Grundablauf

```
plex-enhancer doctor
plex-enhancer preview --artist "Jennifer Rush" --album "Credo"
plex-enhancer review --artist "Jennifer Rush" --album "Credo"
plex-enhancer apply --artist "Jennifer Rush" --album "Credo"
```

Kapitel 3

2. Installation

Dieses Kapitel beschreibt die Installation für Personen ohne Programmiererfahrung.

3.1 2.1 Voraussetzungen

Sie benötigen:

- Python 3.12 oder neuer
- Git
- eine Plex-Musikbibliothek
- Plex-URL und Plex-Token
- optional einen OpenAI API Key

3.2 2.2 Python installieren

3.2.1 macOS

```
brew install python  
python3 --version
```

3.2.2 Windows

Installieren Sie Python von <https://www.python.org/downloads/>. Aktivieren Sie während der Installation die Option **Add Python to PATH**.

Prüfung:

```
python --version
```

3.2.3 Linux

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git
python3 --version
```

3.3 2.3 Git installieren

macOS:

```
brew install git
```

Windows:

Installieren Sie Git von <https://git-scm.com/download/win>.

Linux:

```
sudo apt install git
```

3.4 2.4 Repository klonen

```
git clone https://github.com/amylasehai0z/plex-music-enhancer.git
cd plex-music-enhancer
```

3.5 2.5 Virtuelle Umgebung erstellen

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3.6 2.6 Abhängigkeiten installieren

```
python -m pip install --upgrade pip
python -m pip install ".[dev,ai,metadata]"
```

3.7 2.7 OpenAI API Key

Für echte KI-Erzeugung:

```
export PLEX_ENHANCER_AI__PROVIDER=openai
export OPENAI_API_KEY="sk-..."
```

Windows PowerShell:

```
$env:PLEX_ENHANCER_AI__PROVIDER="openai"
$env:OPENAI_API_KEY="sk-..."
```

Ohne OpenAI API Key verwendet das Programm standardmäßig **dummy**. Dieser Anbieter eignet sich nur für Tests.

3.8 2.8 Plex Token

Sie benötigen:

- Plex Server URL, zum Beispiel `http://localhost:32400`
- Plex Token

Warnung: Ein Plex Token ist ein Geheimnis. Speichern Sie es nicht in öffentlichen Screenshots oder Issues.

3.9 2.9 Login

```
plex-enhancer login
```

Erwartete Eingaben:

```
Plex server URL:
Plex token:
```

Das Token wird versteckt eingegeben. Bei Erfolg speichert das Programm die Werte in `.env`.

3.10 2.10 Doctor

```
plex-enhancer doctor
```

Doctor prüft Installation, Plex, AI-Konfiguration und Cache.

3.11 2.11 Verifikation

```
plex-enhancer version
```

Ausgabe:

```
plex-enhancer 1.0.0
```

3.12 2.12 Häufige Fehler

Fehler	Lösung
Python wird nicht gefunden	Python installieren oder <code>python3</code> verwenden
Aktivierung der Umgebung schlägt fehl	Terminal neu öffnen und Pfad prüfen
Plex ist nicht erreichbar	URL, Token, Port und Firewall prüfen
OpenAI wird nicht genutzt	<code>PLEX_ENHANCER_AI__PROVIDER=openai</code> setzen

Kapitel 4

3. Erste Schritte

Dieses Kapitel führt durch einen vollständigen Einsteigerworkflow mit einem Album.

Beispiel:

- Künstler: Jennifer Rush
- Album: Credo

4.1 3.1 Vorbereitung

Aktivieren Sie die virtuelle Umgebung:

```
source .venv/bin/activate
```

Windows:

```
.\.venv\Scripts\Activate.ps1
```

4.2 3.2 Doctor ausführen

```
plex-enhancer doctor
```

Prüfen Sie, ob Plex-Verbindung und AI-Anbieter stimmen. Wenn **dummy** angezeigt wird, wird keine echte KI-Anfrage gesendet.

4.3 3.3 Login ausführen

Falls Doctor fehlende Plex-Konfiguration meldet:

```
plex-enhancer login
```

Geben Sie Plex-URL und Token ein.

4.4 3.4 Vorschau erzeugen

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo"
```

Die Ausgabe zeigt:

- generierte Zusammenfassung
- Provider
- Modell
- Prompt-Version
- Warnungen

Für Details:

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo" --verbose
```

4.5 3.5 Review starten

```
plex-enhancer review --artist "Jennifer Rush" --album "Credo"
```

Sie sehen:

1. aktuellen Plex-Text
2. neuen Text
3. Diff
4. Qualitätsprüfung
5. Stilprüfung
6. Faktenprüfung

4.6 3.6 Entscheidung treffen

[A] Apply [E] Edit [S] Skip [Q] Quit

Auswahl	Wirkung
A	sicher anwenden
E	Text bearbeiten
S	überspringen
Q	abbrechen

4.7 3.7 Apply ausführen

Direkt:

```
plex-enhancer apply --artist "Jennifer Rush" --album "Credo"
```

Apply erstellt ein Backup, schreibt den Text, lädt das Album neu und prüft den gespeicherten Wert.

4.8 3.8 Ergebnis prüfen

Nach Apply können Sie das Album in Plex öffnen. Zusätzlich liegen Audit- und Backup-Dateien unter:

```
exports/backups/
exports/audit/
```

4.9 3.9 Häufige Entscheidungen

Situation	Empfehlung
Text ist gut	Apply
Text ist fast gut	Edit
Quellen wirken falsch	Skip und mit <code>match</code> prüfen
Sprache ist falsch	<code>--translate</code> nutzen
Text klingt holprig	<code>--improve</code> nutzen

Kapitel 5

4. Konfiguration

Plex Music Enhancer nutzt Umgebungsvariablen und eine lokale `.env` Datei. Alle projektbezogenen Variablen beginnen mit `PLEX_ENHANCER_`.

5.1 4.1 Konfigurationsdateien

Die Datei `.env` liegt im Projektordner. Sie wird durch `plex-enhancer login` angelegt oder aktualisiert.

Beispiel:

```
PLEX_ENHANCER_PLEX_URL=http://localhost:32400
PLEX_ENHANCER_PLEX_TOKEN=...
PLEX_ENHANCER_AI__PROVIDER=openai
```

5.2 4.2 Umgebungsvariablen

Direkt gesetzte Umgebungsvariablen haben Vorrang vor `.env`.

macOS/Linux:

```
export PLEX_ENHANCER_AI__PROVIDER=openai
```

Windows PowerShell:

```
$env:PLEX_ENHANCER_AI__PROVIDER="openai"
```

5.3 4.3 Plex-Konfiguration

Variable	Bedeutung
PLEX_ENHANCER_PLEX_URL	URL des Plex Servers
PLEX_ENHANCER_PLEX_TOKEN	Plex Token
PLEX_ENHANCER_REQUEST_TIMEOUT_SECONDS	Timeout für Diagnosen

5.4 4.4 Provider-Auswahl

Metadatenanbieter:

- MusicBrainz
- Wikipedia
- Discogs
- Last.fm

Optionale Zugangsdaten:

```
PLEX_ENHANCER_DISCOGS__TOKEN
PLEX_ENHANCER_LASTFM__API_KEY
```

5.5 4.5 Modell-Auswahl

```
PLEX_ENHANCER_AI__PROVIDER=openai
PLEX_ENHANCER_AI__MODEL=gpt-5.5
PLEX_ENHANCER_AI__MAX_PROMPT_CHARACTERS=20000
OPENAI_API_KEY=...
```

PLEX_ENHANCER_AI__MAX_PROMPT_CHARACTERS ist die bevorzugte Einstellung. Die Kurzform AI_PROMPT_MAX_CHARS wird weiterhin aus Umgebungsvariablen und explizit geladenen .env Dateien akzeptiert.

Pro Befehl kann das Modell überschrieben werden:

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo" --model "gpt-5.5"
```

Sehr große Künstlerkontexte werden automatisch auf das konfigurierte Prompt-Budget gekürzt. Strukturierte verifizierte Metadaten bleiben erhalten; lange Wikipedia-, Discogs-, Last.fm- und Plex-Texte werden bei Bedarf an Absatz- oder Satzgrenzen gekürzt.

5.6 4.6 Prompt-Versionen

Prompts liegen unter:

```
prompts/
```

Jede generierte Zusammenfassung speichert:

- Prompt-Name
- Prompt-Version
- Modell
- Anbieter

Das hilft bei Nachvollziehbarkeit und späterer Regeneration.

5.7 4.7 Logging

Global:

```
plex-enhancer --log-level DEBUG doctor
```

Standard:

```
INFO
```

Nutzen Sie `DEBUG` nur zur Fehlersuche.

5.8 4.8 Cache

Standardort:

```
~/plex-enhancer/cache/
```

Status:

```
plex-enhancer cache stats
```

Löschen:

```
plex-enhancer cache clear
```

5.9 4.9 Exportordner

```
exports/  
  audit/  
  backups/  
  context/  
  inspect/  
  library/  
  metadata/  
  previews/
```

Diese Dateien sind hilfreich für Diagnose, Backups und Nachvollziehbarkeit.

5.10 4.10 Secrets

Geheime Werte:

- Plex Token
- OpenAI API Key
- Discogs Token
- Last.fm API Key

Warnung: Secrets nicht in Git committen, nicht in Screenshots zeigen und nicht in Issues posten.

5.11 4.11 Best Practices

- `login` für Plex-Zugangsdaten verwenden.
- OpenAI API Key als Umgebungsvariable setzen.
- Cache nicht unnötig löschen.
- Bei großen Bibliotheken zuerst `benchmark` ausführen.
- Vor Apply immer Review-Ausgabe prüfen.

Kapitel 6

5. CLI-Befehle

Dieses Kapitel beschreibt alle öffentlichen Befehle. Die tatsächliche Befehlsliste kann mit folgendem Befehl angezeigt werden:

```
plex-enhancer --help
```

6.1 5.1 Globale Optionen

Option	Bedeutung
--log-level TEXT	Logging-Level, Standard INFO
--help	Hilfe anzeigen

6.2 5.2 version

Zweck: installierte Version anzeigen.

```
plex-enhancer version
```

Typische Ausgabe:

```
plex-enhancer 1.0.0
```

6.3 5.3 doctor

Zweck: Installation und Konfiguration prüfen.

```
plex-enhancer doctor
```

Mögliche Fehler:

- fehlende Plex-URL
- fehlendes Token
- OpenAI Key fehlt
- Plex nicht erreichbar

Best Practice: vor jeder größeren Sitzung ausführen.

6.4 5.4 serve

Zweck: optionale Weboberfläche und FastAPI-REST-Backend starten.

```
python -m pip install ".[web]"
plex-enhancer serve
```

Optionen:

Option	Bedeutung
<code>--host TEXT</code>	Host-Interface, Standard 127.0.0.1
<code>--port INTEGER</code>	Port, Standard 8080; auch über PLEX_ENHANCER_WEB__PORT setzbar
<code>--reload</code>	Uvicorn-Reload für lokale Entwicklung

Die Weboberfläche ist unter <http://127.0.0.1:8080/> erreichbar. Swagger UI ist unter <http://127.0.0.1:8080/swagger-ui/> erreichbar. Container-Installationen können später weiterhin einen Host-Port wie 1008 auf den internen Port 8080 mappen.

6.5 5.5 login

Zweck: Plex-Zugangsdaten lokal speichern.

```
plex-enhancer login
```

Eingaben:

- Plex Server URL
- Plex Token

6.6 5.6 audit

Zweck: vorhandene Metadatenqualität prüfen.

```
plex-enhancer audit
plex-enhancer audit --export-json
```

Export:

```
exports/audit.json
```

6.7 5.7 plan

Zweck: notwendige Aktionen für Alben planen.

```
plex-enhancer plan --library "Music"
plex-enhancer plan --library "Music" --json
```

Aktionen: CREATE, TRANSLATE, IMPROVE, REVIEW, SKIP.

6.8 5.8 benchmark

Zweck: Performance einer Bibliothek prüfen.

```
plex-enhancer benchmark --library "Music"
plex-enhancer benchmark --library "Music" --json
```

6.9 5.9 capabilities

Zweck: Plex-Metadatenfähigkeiten analysieren.

```
plex-enhancer capabilities
```

Export:

```
exports/capabilities.json
```

6.10 5.10 match

Zweck: MusicBrainz Release Group finden.

```
plex-enhancer match --artist "Jennifer Rush" --album "Credo"
plex-enhancer match --artist "Jennifer Rush" --album "Credo" --json
```

6.11 5.11 scan

Zweck: Plex-Musikbibliothek lesen.

```
plex-enhancer scan
plex-enhancer scan --export-json
plex-enhancer scan artists --export-json
plex-enhancer scan albums --export-json
```

Exporte:

- exports/libraries.json
- exports/artists.json
- exports/albums.json

6.12 5.12 inspect

Zweck: Plex-Objekte detailliert untersuchen.

```
plex-enhancer inspect library --name "Music"
plex-enhancer inspect artist --name "Jennifer Rush"
plex-enhancer inspect album --name "Credo"
plex-enhancer inspect track --name "Titel"
```

Optionen:

Option	Bedeutung
--id	Rating Key oder ID
--name	Name/Titel
--json	JSON ausgeben
--save	JSON speichern

6.13 5.13 probe write

Zweck: Schreibfähigkeit prüfen.

```
plex-enhancer probe write --artist "Jennifer Rush" --album "Credo"
plex-enhancer probe write --artist "Jennifer Rush" --album "Credo" --execute
```

Ohne --execute wird nicht geschrieben.

6.14 5.14 metadata album

Zweck: normalisierte Album-Metadaten erzeugen.

```
plex-enhancer metadata album --artist "Jennifer Rush" --album "Credo"
plex-enhancer metadata album --artist "Jennifer Rush" --album "Credo" --json
plex-enhancer metadata album --artist "Jennifer Rush" --album "Credo" --save
```

6.15 5.15 context album

Zweck: vollständigen AlbumContext erzeugen.

```
plex-enhancer context album --artist "Jennifer Rush" --album "Credo"
plex-enhancer context album --artist "Jennifer Rush" --album "Credo" --json
plex-enhancer context album --artist "Jennifer Rush" --album "Credo" --save
```

6.16 5.16 preview

Zweck: Textvorschau erzeugen.

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo"
```

Optionen:

Option	Bedeutung
--provider	AI-Anbieter überschreiben
--model	Modell überschreiben
--json	JSON ausgeben
--save	Vorschau speichern
--verbose	Details anzeigen
--translate	vorhandenen Text übersetzen
--improve	deutschen Text verbessern

6.17 5.17 preview artist

```
plex-enhancer preview artist --artist "Jennifer Rush"
plex-enhancer preview artist --artist "Jennifer Rush" --json
plex-enhancer preview artist --artist "Jennifer Rush" --verbose
plex-enhancer preview artist --artist "Jennifer Rush" --save
```

`--verbose` zeigt die vollständige Künstlerdiagnose mit Plex-Biografie, MusicBrainz, Wikipedia, Discogs, Last.fm, Faktenprüfung, Stilbewertung, Qualitätsanalyse und Prompt-Daten. `--save` speichert die Vorschau unter `exports/previews/artists/`.

Die Diagnose unterscheidet `available`, `missing` und `unknown`. Karrierejahre werden nicht aus Geburtsdaten abgeleitet. Discogs wird nur angezeigt, wenn es zusätzliche Informationen liefert; andernfalls steht dort `No additional artist information available..` Außerdem zeigt `--verbose` das Prompt-Budget, die ursprüngliche und gekürzte Promptgröße sowie die Beiträge der einzelnen Quellen.

6.18 5.18 review album

```
plex-enhancer review album --artist "Jennifer Rush" --album "Credo"
plex-enhancer review album --artist "Jennifer Rush" --album "Credo" --provider openai
plex-enhancer review album --artist "Jennifer Rush" --album "Credo" --improve
```

Optionen:

- `--provider`
- `--model`
- `--json`
- `--translate`
- `--improve`

Die ältere Form `plex-enhancer review --artist ... --album ...` bleibt weiterhin gültig.

6.19 5.19 review artist

```
plex-enhancer review artist --artist "Jennifer Rush"
plex-enhancer review artist --artist "Jennifer Rush" --provider openai
```

Während interaktiver Reviews entstehen temporäre Diagnose-Dateien:

- `/tmp/openai_prompt.txt` enthält exakt den an OpenAI gesendeten Prompt.
- `/tmp/openai_prompt_meta.json` enthält Provider, Modell, Ziel, Prompt-Länge und Budgetdaten.
- `/tmp/plex_review.log` enthält Review-Ausgabe, QA, Stilprüfung, Verifikation, Token-Nutzung und Kontext.

6.20 5.20 apply

```
plex-enhancer apply --artist "Jennifer Rush" --album "Credo"
plex-enhancer apply --artist "Jennifer Rush" --album "Credo" --json
```

Optionen:

- --provider
- --model
- --json
- --translate
- --improve
- --force

6.21 5.21 apply artist

```
plex-enhancer apply artist --artist "Jennifer Rush"
```

6.22 5.22 batch review

```
plex-enhancer batch review --library "Music" --missing-only --limit 25
plex-enhancer batch review --library "Music" --resume
```

6.23 5.23 library

```
plex-enhancer library plan --library "Music"
plex-enhancer library review --library "Music"
plex-enhancer library resume --library "Music"
plex-enhancer library apply --library "Music"
plex-enhancer library report --library "Music" --export-json
```

6.24 5.24 cache

```
plex-enhancer cache stats
plex-enhancer cache list
plex-enhancer cache clear
```

6.25 5.25 Fehlerbehebung bei Befehlen

Symptom	Lösung
Befehl unbekannt	<code>plex-enhancer --help</code> prüfen
Option unbekannt	<code>plex-enhancer <befehl> --help</code> prüfen
Plex-Konfiguration fehlt	<code>plex-enhancer login</code>
Album fehlt	Schreibweise und Bibliothek prüfen
Apply blockiert	Review-Qualität prüfen

Kapitel 7

6. Workflows

Dieses Kapitel beschreibt komplette Arbeitsabläufe.

7.1 6.1 Einzelnes Album

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo"  
plex-enhancer review --artist "Jennifer Rush" --album "Credo"  
plex-enhancer apply --artist "Jennifer Rush" --album "Credo"
```

Empfehlung: Verwenden Sie zuerst **preview**, bevor Sie **apply** nutzen.

7.2 6.2 Einzelner Künstler

```
plex-enhancer preview artist --artist "Jennifer Rush"  
plex-enhancer preview artist --artist "Jennifer Rush" --verbose  
plex-enhancer preview artist --artist "Jennifer Rush" --save  
plex-enhancer review artist --artist "Jennifer Rush"  
plex-enhancer apply artist --artist "Jennifer Rush"
```

Die Künstler-Preview erzeugt eine längere deutsche Biografie mit Karriereüberblick, musikalischem Stil und belegter Bedeutung. Mit **--verbose** sehen Sie Quellen, Faktenprüfung, Qualitätsbewertung und Stilanalyse. Mit **--save** wird das vollständige Preview-Dokument unter `exports/previews/artists/` gespeichert.

7.3 6.3 Ganze Bibliothek

```
plex-enhancer library plan --library "Music"
plex-enhancer library review --library "Music"
plex-enhancer library apply --library "Music"
plex-enhancer library report --library "Music" --export-json
```

7.4 6.4 Übersetzung

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo" --translate
plex-enhancer review --artist "Jennifer Rush" --album "Credo" --translate
```

Übersetzung nutzt den vorhandenen Plex-Text als Hauptquelle.

7.5 6.5 Verbesserung vorhandener deutscher Texte

```
plex-enhancer preview --artist "Jennifer Rush" --album "Credo" --improve
plex-enhancer review --artist "Jennifer Rush" --album "Credo" --improve
```

7.6 6.6 Preview Workflow

```
flowchart TD
    A["Album in Plex suchen"] --> B["Kontext sammeln"]
    B --> C["Prompt bauen"]
    C --> D["AI-Anbieter aufrufen"]
    D --> E["Qualität prüfen"]
    E --> F["Vorschau anzeigen"]
```

7.7 6.7 Review Workflow

```
flowchart TD
    A["Current Summary"] --> B["Generated Summary"]
    B --> C["Diff"]
    C --> D["Quality Checks"]
    D --> E["Benutzerentscheidung"]
```

7.8 6.8 Apply Workflow

```
flowchart TD
    A["Review bestanden"] --> B["Backup"]
    B --> C["Plex schreiben"]
    C --> D["Reload"]
    D --> E["Verifikation"]
    E --> F["Audit"]
```

7.9 6.9 Batch Processing

```
plex-enhancer batch review --library "Music" --missing-only --limit 50
```

Nutzen Sie Batch, wenn Sie mehrere Alben nacheinander prüfen möchten.

7.10 6.10 Library Mode

Der Library-Modus speichert Sitzungen und erlaubt das Fortsetzen.

```
plex-enhancer library resume --library "Music"
```

7.11 6.11 Qualitätssicherung

Vor Apply prüft das System:

- Sprache
- Länge
- Markdown
- Bullet-Listen
- offene Template- oder Testtexte
- Stil
- Faktenabdeckung

7.12 6.12 Rollback-Strategie

v1.0 speichert Backups vor Apply. Ein automatischer Rollback-Befehl ist nicht der Hauptworkflow. Wenn Sie zurücksetzen müssen, nutzen Sie die Backup-Datei unter `exports/backups/` als Quelle.

7.13 6.13 Best Practices

- Kleine Testläufe vor großen Bibliotheken.
- Cache nicht unnötig löschen.
- `--verbose` bei unklaren Quellen verwenden.
- JSON-Exporte bei Fehlern aufbewahren.
- Review nie überspringen, wenn der Stil neu eingerichtet wurde.

Kapitel 8

7. KI und Editorial Engine

Plex Music Enhancer nutzt KI nicht isoliert. Die KI erhält einen geprüften Kontext und klare Schreibregeln.

8.1 7.1 Gesamtarchitektur

```
Plex
↓
Metadata Providers
↓
MusicBrainz
↓
Wikipedia
↓
Discogs
↓
Last.fm
↓
Context Builder
↓
Editorial Style Engine
↓
GPT-5.5
↓
Review Engine
↓
Apply
```

8.2 7.2 Plex

Plex liefert den lokalen Ausgangspunkt:

- Künstler
- Album
- Jahr
- vorhandene Beschreibung
- Genres
- Rating Key

8.3 7.3 Metadata Providers

Externe Quellen ergänzen Fakten. Optional nicht verfügbare Quellen blockieren den Ablauf nicht.

8.4 7.4 MusicBrainz

MusicBrainz liefert stabile IDs und Release-Informationen. Es ist besonders wichtig für korrekte Albumzuordnung.

8.5 7.5 Wikipedia

Wikipedia liefert enzyklopädischen Kontext. Wenn kein Artikel existiert, wird der Workflow fortgesetzt.

8.6 7.6 Discogs

Discogs kann Labels, Katalognummern, Credits und Produktionsinformationen liefern. Dafür ist ein Token erforderlich.

8.7 7.7 Last.fm

Last.fm kann Tags, Hörerzahlen und Biografieinformationen liefern. Dafür ist ein API Key erforderlich.

8.8 7.8 Context Builder

Der Context Builder erstellt:

- AlbumContext
- ArtistContext

Diese Modelle sind die strukturierte Grundlage für Prompt und Qualitätssicherung.

8.9 7.9 Editorial Style Engine

Die Editorial Style Engine analysiert:

- Satzanfänge
- Wortwiederholungen
- Lesbarkeit
- typische KI-Floskeln
- Listenstil
- passive Sprache

8.10 7.10 GPT-5.5 oder anderes Modell

Das Modell erzeugt die finale Formulierung. Es erhält keine freie Aufgabe, sondern einen Prompt mit Fakten und Grenzen.

8.11 7.11 Prompt Engineering

Prompts enthalten:

- Ziel: deutscher enzyklopädischer Stil
- gewünschte Länge
- Faktenregeln
- Verbote gegen erfundene Charts, Preise oder Kritiken
- Variablen aus dem Kontext

8.12 7.12 Prompt-Versionen

Prompt-Versionen werden gespeichert, damit nachvollziehbar bleibt, welche Vorlage einen Text erzeugt hat.

8.13 7.13 Warum Texte factual bleiben

Das System nutzt:

- Quellenkontext
- Fact Verification
- Prompt-Regeln
- Review
- QA

Wichtig: Das System reduziert Halluzinationen, ersetzt aber keine menschliche Prüfung.

Kapitel 9

8. Review-System

Das Review-System schützt vor ungeprüften Änderungen.

9.1 8.1 Quality Score

Der Quality Score bewertet den generierten Text von 0 bis 100.

Score	Bedeutung
90-100	sehr gut
80-89	gut
70-79	prüfen
unter 70	problematisch

9.2 8.2 Checks

Geprüft werden:

- nicht leer
- Deutsch erkennbar
- kein Markdown
- keine Bullet-Listen
- keine offenen Template- oder Testtexte

Diese Punkte sind kritische Validierungen. Sie blockieren Apply.

9.3 8.3 Quality Summary

Die Review-Ausgabe zeigt zusätzlich:

Feld	Bedeutung
Critical validation	harte Regeln zu Sprache, Leertext, Format und Fakten
Editorial validation	redaktionelle Qualität und Stil
Publishable	ob der Text angewendet werden darf

PASS bedeutet, dass keine Probleme gefunden wurden. WARNINGS bedeutet, dass Apply erlaubt ist, aber stilistische Verbesserungen möglich sind. FAILED bedeutet, dass Apply blockiert wird.

9.4 8.4 Language Detection

Die Spracherkennung ist eine Heuristik. Sie prüft typische deutsche Wörter und Zeichen.

9.5 8.5 Length Validation

Zu kurze Texte enthalten meist zu wenig Kontext. Zu lange Texte passen schlecht in Plex und wirken schnell überladen.

9.6 8.6 Editorial Validation

Die Editorial-Prüfung achtet auf:

- Wiederholungen
- schwache Übergänge
- generische Satzanfänge
- abruptes Ende
- Listenstil

Diese Hinweise sind Empfehlungen. Sie blockieren Apply nicht, wenn der Text insgesamt publizierbar ist.

9.7 8.7 Apply-Policy

Apply ist erlaubt, wenn:

- alle kritischen Validierungen bestehen
- keine faktischen Konflikte vorliegen

- das Verifikationsvertrauen ausreichend ist
- der redaktionelle Score mindestens 85 beträgt oder das Qualitätslevel GOOD, VERY GOOD oder EXCELLENT ist

Wenn nur redaktionelle Warnungen vorhanden sind, zeigt das Programm:

```
Editorial warnings detected.
The generated summary is considered publishable.
You may still Apply this summary.

Continue? [Y/n]
```

Wenn der Score zu niedrig ist, wird Apply mit dieser Meldung blockiert:

```
Generated summary does not yet meet the required editorial quality.
```

9.8 8.8 Diff Viewer

Der Diff zeigt Unterschiede:

```
- alter Text
+ neuer Text
```

9.9 8.9 Approval Process

```
[A] Apply  [E] Edit  [S] Skip  [Q] Quit
```

Taste	Bedeutung
A	anwenden
E	bearbeiten
S	überspringen
Q	beenden

9.10 8.10 Manuelle Bearbeitung

Bei E öffnet sich der konfigurierte Terminaleditor. Nach dem Speichern wird erneut geprüft.

9.11 8.11 Warum Summaries scheitern können

Gründe:

- leere Ausgabe
- falsche Sprache
- zu kurz
- Markdown
- Liste statt Fließtext
- schwacher Stil
- unklare Faktenlage

9.12 8.12 Typischer Review Workflow

```
plex-enhancer review album --artist "Jennifer Rush" --album "Credo" --provider openai
plex-enhancer review artist --artist "Jennifer Rush" --provider openai
```

1. Text lesen.
2. Diff prüfen.
3. Qualitätswarnungen ansehen.
4. Bei Bedarf bearbeiten.
5. Apply nur bei plausibler Ausgabe.

9.13 8.13 Debug-Dateien

Für die Fehlersuche werden während interaktiver Reviews temporäre Dateien unter `/tmp` erzeugt und bei jedem Lauf überschrieben:

Datei	Inhalt
<code>/tmp/openai_prompt.txt</code>	exakt der an OpenAI gesendete Prompt
<code>/tmp/openai_prompt_meta.json</code>	Zeitstempel, Provider, Modell, Ziel, Prompt-Länge und Budgetdaten
<code>/tmp/plex_review.log</code>	Review-Ausgabe, Diff, QA, Stilprüfung, Verifikation, Token-Nutzung und Kontext

Der Review-Log enthält außerdem `PROMPT BUDGET`, `USED SOURCES`, `PROMPT DECISIONS`, `EVIDENCE RANKING`, `PROMPT QUALITY`, `EVIDENCE COVERAGE`, `EDITORIAL BALANCE`, `EDITORIAL COVERAGE`, `PROMPT UTILIZATION`, `PROMPT META` und `RESPONSE META`. Diese Abschnitte zeigen, welche Quellen im Prompt vertreten waren, welcher Anteil des Budgets auf Wikipedia, Discogs, Last.fm, aktuelle Plex-Biografie, Regeln und Sicherheitsanweisungen entfiel, welche Evidenz adaptiv gekürzt wurde und welche vorhandenen Themen die generierte Biografie noch nicht genutzt hat. `PROMPT QUALITY` fasst zusätzlich Prompt-Redundanz, Quellenbalance, historische Abdeckung

und Prompt Efficiency zusammen. **EVIDENCE RANKING** bewertet die Evidenz nach historischem Wert, wichtigen Werken, Karriereentwicklung, Legacy, internationaler Anerkennung, Einzigartigkeit und Informationsdichte. **EVIDENCE COVERAGE** zeigt, wie viel hochwertige Evidenz in der Ausgabe angekommen ist, während **EDITORIAL BALANCE** die erzählerische Ausgewogenheit der Biografie prüft. Die Prompt-Kompression entfernt bevorzugt Wiederholungen und redundante Erfolgsaussagen, bevor historisch relevante Informationen gekürzt werden.

Kapitel 10

9. Provider

Provider sind Datenquellen. Sie liefern Fakten, aber schreiben nichts nach Plex.

10.1 9.1 Plex

Aspekt	Beschreibung
Zweck	lokale Bibliotheksdaten lesen und Ziel für Apply
Metadaten	Titel, Künstler, Jahr, Genres, Zusammenfassung
Zuverlässigkeit	hoch für lokale Bibliothek
Grenze	vorhandene Daten können unvollständig sein

10.2 9.2 MusicBrainz

Aspekt	Beschreibung
Zweck	stabile Identifikation von Künstlern und Alben
Metadaten	MBIDs, Release Groups, Daten, Genres, Credits
Zuverlässigkeit	hoch für strukturierte Musikdaten
Grenze	nicht jedes Release ist vollständig gepflegt

10.3 9.3 Wikipedia

Aspekt	Beschreibung
Zweck	enzyklopädischer Kontext
Metadaten	Titel, Sprache, Auszug, URL, Thumbnail

Aspekt	Beschreibung
Zuverlässigkeit	gut, abhängig vom Artikel
Grenze	Artikel können fehlen oder unvollständig sein

10.4 9.4 Discogs

Aspekt	Beschreibung
Zweck	Release- und Creditdaten
Metadaten	Label, Katalognummer, Credits, Formate
Zuverlässigkeit	gut für physische Releases
Grenze	Token erforderlich, Rollen können uneinheitlich sein

10.5 9.5 Last.fm

Aspekt	Beschreibung
Zweck	Community-Kontext
Metadaten	Tags, Biografien, Hörerzahlen
Zuverlässigkeit	unterstützend
Grenze	Communitydaten sind nicht immer neutral

10.6 9.6 OpenAI

Aspekt	Beschreibung
Zweck	Textgenerierung
Eingabe	generierter Prompt
Ausgabe	generierte Zusammenfassung
Grenze	Ausgabe muss geprüft werden

10.7 9.7 Fehlerbehandlung

Providerfehler werden isoliert. Ein fehlender optionaler Provider soll den gesamten Workflow nicht unnötig abbrechen.

Kapitel 11

10. Cache

Der Cache speichert Providerdaten lokal.

11.1 10.1 Zweck

Der Cache reduziert:

- Wartezeit
- Netzwerkanfragen
- Rate-Limit-Probleme
- wiederholte identische Provideraufrufe

11.2 10.2 Speicherorte

```
~/.plex-enhancer/cache/  
~/.plex-enhancer/processing.sqlite3
```

11.3 10.3 Lebensdauer

Standard:

```
30 Tage
```

11.4 10.4 Cache prüfen

```
plex-enhancer cache stats  
plex-enhancer cache list
```

11.5 10.5 Cache löschen

```
plex-enhancer cache clear
```

Warnung: Danach dauert der nächste Lauf länger.

11.6 10.6 Cache neu aufbauen

Führen Sie Preview, Plan oder Library-Workflows erneut aus. Der Cache füllt sich automatisch.

11.7 10.7 Wann löschen?

Sinnvoll bei:

- veralteten Providerdaten
- beschädigten Cachedateien
- Tests mit neuen Quellen
- unerklärlichen Wiederholungsfehlern

Nicht sinnvoll vor jedem Lauf.

11.8 10.8 Performance-Beispiele

Situation	Erwartung
kalter Cache	langsamer, viele Netzwerkanfragen
warmer Cache	schneller, weniger Netzwerkanfragen
große Bibliothek	zuerst Benchmark und Plan verwenden

Kapitel 12

11. Fehlersuche

Dieses Kapitel ist nach Symptomen geordnet.

12.1 11.1 Keine Verbindung zu Plex

Symptome:

- Unable to connect to Plex
- Doctor meldet Plex-Fehler

Ursachen:

- Server läuft nicht
- URL falsch
- Firewall blockiert
- Token falsch

Lösung:

```
plex-enhancer login  
plex-enhancer doctor
```

12.2 11.2 Authentifizierung fehlgeschlagen

Prüfen Sie Plex Token und Konto. Erzeugen Sie bei Bedarf ein neues Token.

12.3 11.3 Album nicht gefunden

Ursachen:

- Titel anders geschrieben
- Künstlername anders geschrieben
- Album liegt nicht in Musikbibliothek

Hilfen:

```
plex-enhancer scan albums --export-json
plex-enhancer inspect album --name "Credo"
```

12.4 11.4 MusicBrainz nicht verfügbar

Nutzen Sie:

```
plex-enhancer match --artist "Jennifer Rush" --album "Credo"
```

Wenn kein Match gefunden wird, prüfen Sie Schreibweise und Jahr.

12.5 11.5 Wikipedia nicht verfügbar

Das ist nicht immer kritisch. Manche Alben oder Künstler haben keinen passenden Artikel.

12.6 11.6 OpenAI Timeout

Lösungen:

- später erneut versuchen
- Timeout erhöhen
- Netzwerk prüfen

```
export PLEX_ENHANCER_AI__TIMEOUT_SECONDS=60
```

12.7 11.7 Rate Limits

Reduzieren Sie parallele Worker:

```
export PLEX_ENHANCER_PERFORMANCE__MAX_WORKERS=2
```

12.8 11.8 Providerfehler

Optionale Provider können ausfallen. Prüfen Sie Zugangsdaten und Cache.

12.9 11.9 Cache-Probleme

```
plex-enhancer cache stats  
plex-enhancer cache clear
```

12.10 11.10 Review fehlgeschlagen

Ursachen:

- Text zu kurz
- falsche Sprache
- Markdown
- Bullet-Liste
- offene Template- oder Testtexte

Lösung:

- im Review E wählen
- Prompt und Quellen prüfen

12.11 11.11 Apply fehlgeschlagen

Prüfen Sie:

- Backup vorhanden?
- Write successful?
- Verification passed?
- Audit stored?

Die Ausgabe des Apply-Workflows zeigt diese Punkte als Tabelle.

12.12 11.12 Konfigurationsfehler

Nutzen Sie:

```
plex-enhancer doctor
```

12.13 11.13 API Key fehlt

Setzen Sie:

```
export OPENAI_API_KEY="sk-..."
```

12.14 11.14 Netzwerkprobleme

Prüfen:

- Internetverbindung
- DNS
- Proxy
- Firewall
- Providerstatus

Kapitel 13

12. Häufig gestellte Fragen

Dieses Kapitel beantwortet typische Fragen aus Einrichtung, täglicher Nutzung und Fehleranalyse.

13.1 Grundlagen

13.1.1 1. Was macht Plex Music Enhancer?

Plex Music Enhancer sammelt vorhandene Plex-Metadaten, ergänzt sie mit externen Quellen und kann daraus deutschsprachige Album- und Künstlertexte erzeugen. Änderungen an Plex passieren nur im Apply-Workflow.

13.1.2 2. Verändert das Programm meine Plex-Bibliothek automatisch?

Nein. Preview, Review, Scan, Plan, Audit, Context, Match, Inspect, Cache und die Provider-Abfragen sind lesend. Geschrieben wird erst mit `plex-enhancer apply` oder einem bestätigten Apply-Schritt.

13.1.3 3. Brauche ich Programmierkenntnisse?

Nein. Sie benötigen nur ein Terminal, Python, die Plex-Zugangsdaten und bei echter KI-Nutzung einen OpenAI API Key.

13.1.4 4. Kann ich das Programm ohne OpenAI nutzen?

Ja. Viele Befehle funktionieren ohne OpenAI, zum Beispiel `doctor`, `scan`, `audit`, `plan`, `match`, `context`, `inspect` und `cache`.

13.1.5 5. Warum gibt es einen DummyProvider?

Der DummyProvider erzeugt deterministische Testtexte. Er ist nützlich für Installation, Tests und Abläufe ohne echte KI-Kosten.

13.1.6 6. Warum nutzt Preview nicht OpenAI, obwohl ein API Key gesetzt ist?

Prüfen Sie `plex-enhancer doctor`. Wenn `ai.provider` auf `dummy` steht, bleibt der Dummy-Provider aktiv, auch wenn `OPENAI_API_KEY` vorhanden ist.

13.1.7 7. Welche Python-Version brauche ich?

Python 3.12 oder neuer.

13.1.8 8. Funktioniert das auf macOS, Windows und Linux?

Das Projekt ist als Python-CLI plattformübergreifend angelegt. Pfade, virtuelle Umgebungen und Shell-Befehle unterscheiden sich je nach System leicht.

13.1.9 9. Muss Plex lokal laufen?

Nein. Die konfigurierte Plex URL muss vom Rechner erreichbar sein, auf dem Plex Music Enhancer läuft.

13.1.10 10. Funktioniert das mit Plexamp?

Plexamp nutzt dieselbe Plex-Bibliothek. Plex Music Enhancer arbeitet gegen den Plex Server, nicht direkt gegen Plexamp.

13.2 Installation und Einrichtung

13.2.1 11. Wie prüfe ich, ob die Installation funktioniert?

Führen Sie `plex-enhancer version` und danach `plex-enhancer doctor` aus.

13.2.2 12. Was macht `plex-enhancer login`?

Der Befehl fragt die Plex Server URL und das Plex Token ab, prüft die Verbindung und speichert die Werte in `.env`, ohne das Token auszugeben.

13.2.3 13. Wird mein Plex Token angezeigt?

Nein. Das Token wird verborgen abgefragt und nicht in der Ausgabe angezeigt.

13.2.4 14. Darf ich .env in Git speichern?

Nein. `.env` enthält Zugangsdaten und gehört nicht in ein öffentliches Repository.

13.2.5 15. Woher bekomme ich mein Plex Token?

Das Plex Token wird über Plex selbst ermittelt. Die genaue Vorgehensweise hängt von Ihrer Plex-Oberfläche ab.

13.2.6 16. Was bedeutet “configuration missing”?

Meist fehlen Plex URL, Plex Token oder ein erforderlicher API Key für einen aktivierten Provider.

13.2.7 17. Warum schlägt doctor fehl?

Häufige Gründe sind falsche URL, ungültiges Token, nicht erreichbarer Server oder fehlende KI-Konfiguration.

13.2.8 18. Kann ich mehrere Plex Server nutzen?

Die CLI verwendet die aktuell gesetzte Konfiguration. Für mehrere Server können Sie getrennte Arbeitsverzeichnisse oder unterschiedliche Umgebungsvariablen verwenden.

13.2.9 19. Welche Datei ist wichtiger: .env oder Umgebungsvariablen?

Beide werden über die Anwendungskonfiguration gelesen. Für reproduzierbare Skripte sind explizite Umgebungsvariablen oft klarer.

13.2.10 20. Warum wird mein OpenAI Key nicht erkannt?

Prüfen Sie, ob `OPENAI_API_KEY` in derselben Shell gesetzt ist, in der Sie `plex-enhancer` ausführen.

13.3 Plex und Bibliotheken

13.3.1 21. Welche Plex-Bibliotheken werden verarbeitet?

Die Workflows betrachten Musikbibliotheken. Andere Bibliothekstypen werden für Musikfunktionen ignoriert.

13.3.2 22. Wie finde ich heraus, welche Musikbibliotheken erkannt werden?

Nutzen Sie `plex-enhancer scan`.

13.3.3 23. Wie exportiere ich Bibliotheksstatistiken?

Führen Sie `plex-enhancer scan --export-json` aus. Die Datei landet unter `exports/libraries.json`.

13.3.4 24. Warum wird ein Album nicht gefunden?

Titel oder Künstler können in Plex anders geschrieben sein. Prüfen Sie mit `plex-enhancer scan albums --export-json` oder `plex-enhancer inspect album --name "Titel"`.

13.3.5 25. Was ist ein Rating Key?

Ein Rating Key ist eine Plex-interne Kennung für ein Objekt.

13.3.6 26. Kann ich nach ID statt nach Name suchen?

Inspect-Befehle unterstützen `--id <ratingKey>` oder `--name "<Titel>"`.

13.3.7 27. Werden Tracks verändert?

Die beschriebenen Apply-Workflows schreiben Zusammenfassungen für Alben oder Künstler. Track-Metadaten werden im Handbuch nicht als Schreibziel beschrieben.

13.3.8 28. Kann Plex Music Enhancer Playlists bearbeiten?

Nein. Das Handbuch dokumentiert keine Playlist-Bearbeitung.

13.3.9 29. Kann ich nur eine bestimmte Bibliothek planen?

Ja, viele Bibliotheksbefehle unterstützen `--library`.

13.3.10 30. Was passiert bei doppelten Albumtiteln?

Der Workflow muss ein eindeutiges Objekt finden. Bei Mehrdeutigkeiten sollten Sie die Plex-Metadaten prüfen und genauer suchen.

13.4 KI und Texte

13.4.1 31. Warum erfindet GPT keine Chartpositionen?

Der Prompt verbietet nicht belegte Fakten. Chartpositionen werden nur verwendet, wenn sie im Kontext vorhanden sind.

13.4.2 32. Warum sind Produzenten manchmal nicht enthalten?

Produzenten erscheinen nur, wenn sie aus den vorhandenen Quellen geliefert und in den Kontext übernommen wurden.

13.4.3 33. Warum sind die Texte konservativ?

Das System bevorzugt belegbare Aussagen gegenüber spekulativen Formulierungen.

13.4.4 34. Warum ist ein Text kürzer als erwartet?

Wenn wenig belastbarer Kontext vorhanden ist, soll die KI keine Lücken füllen.

13.4.5 35. Warum klingt ein Text manchmal allgemein?

Das passiert vor allem bei schwacher Quellenlage oder fehlendem Wikipedia-, Discogs- oder MusicBrainz-Kontext.

13.4.6 36. Kann ich einen Text neu erzeugen?

Ja. Führen Sie Preview oder Review erneut aus. Bei echten KI-Anbietern kann das erneut Kosten verursachen.

13.4.7 37. Kann ich manuell bearbeiten?

Ja. Im Review-Workflow kann die generierte Fassung bearbeitet werden.

13.4.8 38. Warum wird Markdown abgelehnt?

Plex-Zusammenfassungen sollen als klare Fließtexte gespeichert werden, nicht als Markdown-Dokumente.

13.4.9 39. Warum werden Bullet-Listen abgelehnt?

Die Qualitätssicherung erwartet enzyklopädische Prosa.

13.4.10 40. Kann ich englische Texte übersetzen?

Ja. Nutzen Sie Preview mit `--translate` oder entsprechende Review- und Apply-Workflows.

13.4.11 41. Kann ich deutsche Texte verbessern?

Ja. Nutzen Sie Preview mit `--improve`.

13.4.12 42. Wird beim Übersetzen zusammengefasst?

Der Übersetzungsprompt ist darauf ausgelegt, Fakten zu erhalten und nicht frei zu kürzen.

13.4.13 43. Was passiert, wenn der vorhandene Text schon gut ist?

Der Planner kann SKIP empfehlen.

13.4.14 44. Was bedeutet REVIEW im Plan?

Der Text ist nicht eindeutig schlecht, aber eine menschliche Entscheidung ist sinnvoll.

13.4.15 45. Warum lehnt Review einen Text ab?

Mögliche Gründe sind falsche Sprache, zu kurze Länge, Listenstil, wiederholende Formulierungen oder offene Testtexte.

13.5 Provider

13.5.1 46. Was ist MusicBrainz?

MusicBrainz ist eine strukturierte Musikdatenbank und besonders hilfreich für MBIDs, Release Groups und Veröffentlichungsdaten.

13.5.2 47. Was ist eine MBID?

Eine MBID ist eine MusicBrainz Identifier, also eine stabile Kennung für Künstler, Releases oder Release Groups.

13.5.3 48. Was ist eine Release Group?

Eine Release Group bündelt verschiedene Veröffentlichungen desselben Albums.

13.5.4 49. Was liefert Wikipedia?

Wikipedia liefert enzyklopädische Auszüge und Links, sofern ein passender Artikel gefunden wird.

13.5.5 50. Warum findet Wikipedia keinen Artikel?

Nicht jedes Album hat einen Artikel. Außerdem können Schreibweise, Sprache und Mehrdeutigkeit eine Rolle spielen.

13.5.6 51. Was liefert Discogs?

Discogs kann Labels, Katalognummern, Formate und Credits liefern, wenn der Provider konfiguriert ist.

13.5.7 52. Was liefert Last.fm?

Last.fm kann Tags, Hörerdaten und biografische Hintergrundinformationen liefern, wenn der Provider konfiguriert ist.

13.5.8 53. Sind Last.fm-Tags objektive Fakten?

Nein. Sie sind Community-Signale und werden als unterstützender Kontext behandelt.

13.5.9 54. Werden Providerfehler abgefangen?

Ja. Optionale Provider sollen den gesamten Workflow nicht stoppen.

13.5.10 55. Muss ich alle Provider konfigurieren?

Nein. Plex und die gewählte KI-Konfiguration sind für die jeweiligen Workflows entscheidend. Zusätzliche Provider verbessern die Quellenlage.

13.6 Cache und Performance

13.6.1 56. Warum gibt es einen Cache?

Der Cache reduziert wiederholte Provider-Abfragen und beschleunigt spätere Läufe.

13.6.2 57. Wo liegt der Cache?

Der lokale Knowledge Cache liegt im Benutzerverzeichnis unter `.plex-enhancer/cache`.

13.6.3 58. Wie sehe ich Cache-Statistiken?

Mit `plex-enhancer cache stats`.

13.6.4 59. Wie liste ich Cache-Einträge?

Mit `plex-enhancer cache list`.

13.6.5 60. Wie lösche ich den Cache?

Mit `plex-enhancer cache clear`.

13.6.6 61. Wann sollte ich den Cache löschen?

Wenn veraltete Providerdaten vermutet werden oder Sie einen frischen Lauf erzwingen möchten.

13.6.7 62. Wird ein fehlgeschlagener Provider dauerhaft gecacht?

Fehlschläge sollen den Cache nicht dauerhaft vergiften. Erfolgreiche Antworten werden bevorzugt gespeichert.

13.6.8 63. Warum ist der erste Lauf langsam?

Beim ersten Lauf müssen Plex und externe Provider abgefragt werden. Spätere Läufe profitieren vom Cache.

13.6.9 64. Warum ist ein großer Bibliothekslauf langsam?

Mehrere hundert oder tausend Alben erzeugen viele Plex-, Provider- und KI-Schritte. Nutzen Sie Limits, Planung und Resume.

13.6.10 65. Kann ich parallelisieren?

Das Projekt enthält Performance-Optionen, aber Provider-Grenzen und Sicherheit sind wichtiger als maximale Geschwindigkeit.

13.7 Review und Apply

13.7.1 66. Was ist der Unterschied zwischen Preview und Review?

Preview zeigt eine Vorschau. Review ergänzt Vergleich, Qualitätsprüfung und Entscheidung.

13.7.2 67. Was ist der Unterschied zwischen Review und Apply?

Review verändert Plex nicht. Apply schreibt eine genehmigte Zusammenfassung mit Backup, Reload und Verifikation.

13.7.3 68. Kann ich Änderungen rückgängig machen?

Apply legt Backups unter `exports/backups/` an. Diese Backups sind die Grundlage für eine manuelle Wiederherstellung.

13.7.4 69. Wo liegen Audit-Dateien?

Audit-Daten des Apply-Workflows liegen unter `exports/audit/`.

13.7.5 70. Was passiert, wenn Verifikation fehlschlägt?

Dann wird kein Erfolg gemeldet. Der Backup bleibt erhalten.

13.7.6 71. Kann Apply den falschen Datensatz ändern?

Apply sucht anhand der übergebenen Angaben. Arbeiten Sie bei Mehrdeutigkeiten vorsichtig und prüfen Sie Preview und Review.

13.7.7 72. Warum gibt es keine automatische Massenänderung ohne Review?

Die Sicherheitsphilosophie verlangt menschliche Kontrolle vor dauerhaften Plex-Änderungen.

13.7.8 73. Kann ich Batch-Verarbeitung abbrechen?

Ja. Interaktive Batch-Workflows bieten Skip und Quit.

13.7.9 74. Was macht Resume?

Resume setzt eine unterbrochene Sitzung anhand gespeicherter Jobdaten fort.

13.7.10 75. Wo sehe ich eine Zusammenfassung eines Bibliothekslaufs?

Mit `plex-enhancer library report`.

13.8 JSON, Exporte und Fehlersuche

13.8.1 76. Wo werden JSON-Dateien gespeichert?

Unter `exports/`, je nach Befehl in passenden Unterordnern.

13.8.2 77. Kann ich JSON-Ausgaben weiterverarbeiten?

Ja. Viele Befehle bieten `--json` oder `--export-json`.

13.8.3 78. Warum unterscheiden sich Rich-Ausgabe und JSON?

Rich ist für Menschen optimiert. JSON enthält strukturierte Daten für Analyse, Archivierung und Automatisierung.

13.8.4 79. Wie finde ich heraus, welcher Prompt verwendet wurde?

Preview und gespeicherte Vorschauen enthalten Prompt-Name und Prompt-Version.

13.8.5 80. Warum stimmen Prompt-Versionen im Text nicht mit alten Ausgaben überein?

Alte Preview- oder Audit-Dateien behalten die damals verwendete Version. Neue Läufe nutzen die aktuelle Vorlage.

13.8.6 81. Was mache ich bei unverständlichen Fehlern?

Starten Sie mit `plex-enhancer doctor`, prüfen Sie danach den konkreten Befehl mit `--help`.

13.8.7 82. Wann soll ich `--verbose` nutzen?

Wenn Sie Providerdiagnosen, Promptdetails oder zusätzliche Ursacheninformationen sehen möchten.

13.8.8 83. Sind Stack Traces normal?

Im normalen Betrieb sollten nutzerfreundliche Fehlermeldungen erscheinen. Technische Details sind vor allem für Debugging gedacht.

13.8.9 84. Kann ich die Dokumentation als PDF bauen?

Ja. Nutzen Sie das Skript unter `docs/pdf/build.sh`, wenn Pandoc installiert ist.

13.8.10 85. Ist das Handbuch auch für gedruckte Nutzung gedacht?

Ja. Die Kapitelstruktur ist bewusst linear und PDF-freundlich.

Kapitel 14

13. Glossar

Dieses Glossar erklärt Begriffe, die in Plex Music Enhancer und im Handbuch häufig vorkommen.

Begriff	Erklärung
Album Context	Strukturierter Datensatz, der Plex-, Provider-, Prüf- und Kontextinformationen zu einem Album bündelt.
Artist Context	Strukturierter Datensatz für Künstlerbiografien und Künstlerinformationen.
Apply	Workflow, der eine genehmigte Zusammenfassung nach Backup und Verifikation in Plex schreibt.
Audit	Lesende Analyse der vorhandenen Plex-Metadaten.
Backup	Sicherung des alten Plex-Textes vor einer Schreiboperation.
Batch	Interaktiver Ablauf für mehrere Alben.
Cache	Lokaler Zwischenspeicher für Providerdaten.
CLI	Command Line Interface, also die Bedienung über Terminalbefehle.
Context Builder	Teil der Pipeline, der aus Plex und Providerdaten einen einheitlichen Kontext erstellt.
Discogs	Externe Musikdatenbank mit Labels, Katalognummern, Formaten und Credits.
Doctor	Diagnosebefehl für Installation, Konfiguration, Plex und KI.
DummyProvider	Testanbieter, der deterministische Texte ohne Netzwerk erzeugt.

Begriff	Erklärung
Editorial Composer	Schicht, die Fakten in sinnvolle Schreibanweisungen und Prioritäten überführt.
Editorial Style Engine	Analyse- und Polierschicht für deutschen enzyklopädischen Stil.
Export	JSON-Datei, die aus einem Befehl unter exports/ geschrieben wird.
Fact Verification	Bewertung der Zuverlässigkeit einzelner Fakten anhand vorhandener Quellen.
GeneratedSummary	Modell für einen erzeugten Text inklusive Provider, Modell, Prompt und Metadaten.
Halluzination	Nicht belegte oder erfundene Aussage eines Sprachmodells.
Improve	Workflow zur sprachlichen Verbesserung eines vorhandenen deutschen Textes.
Inspect	Lesender Befehl zur Anzeige eines Plex-Objekts mit Attributen, Medien, Bildern und Kindern.
Knowledge Cache	Lokaler Cache für Künstler- und Albumwissen aus externen Quellen.
Last.fm	Provider für Community-Tags, Hörerzahlen und biografische Hintergrunddaten.
Library	Plex-Bibliothek, zum Beispiel eine Musikbibliothek.
Library Workflow	Plan-, Review-, Apply-, Resume- und Report-Ablauf für eine ganze Bibliothek.
Match	Zuordnung eines Plex-Albuns zu MusicBrainz-Daten.
MBID	MusicBrainz Identifier, eine stabile ID für Musikobjekte.
Metadata	Beschreibende Daten wie Titel, Jahr, Genre, Label, Zusammenfassung oder Credits.
MusicBrainz	Strukturierte Musikdatenbank für Künstler, Releases und Release Groups.
OpenAIProvider	KI-Anbieter, der über die OpenAI Python SDK verwendet wird.
Pipeline	Abfolge von Verarbeitungsschritten von Plex über Provider bis zum finalen Kontext.
Planner	Analysekomponente, die CREATE, TRANSLATE, IMPROVE, REVIEW oder SKIP empfiehlt.

Begriff	Erklärung
Preview	Vorschau auf generierte Inhalte ohne Plex-Änderung.
Prompt	Textvorlage mit Anweisungen und Variablen für das KI-Modell.
Prompt Engine	System zum Laden, Prüfen und Rendern von Prompt-Vorlagen.
Prompt Version	Versionsnummer einer Prompt-Vorlage, wichtig für Nachvollziehbarkeit.
Provider	Datenquelle oder KI-Anbieter, zum Beispiel MusicBrainz, Wikipedia, Discogs, Last.fm oder OpenAI.
QA	Quality Assurance, also Qualitätsprüfung des erzeugten Textes.
Rating Key	Plex-interne Kennung für ein Objekt.
Release	Eine konkrete Veröffentlichung eines Albums.
Release Group	MusicBrainz-Gruppe mehrerer Veröffentlichungen desselben Albums.
Resume	Fortsetzen eines unterbrochenen Jobs.
Review	Interaktive Prüfung mit Vergleich, Qualitätsanalyse und Entscheidung.
Rich	Terminalbibliothek für Tabellen, Panels und formatierte Ausgaben.
Scan	Lesende Erfassung von Plex-Bibliotheken, Künstlern oder Alben.
Style Analysis	Analyse von Satzanfängen, Wiederholungen, Lesbarkeit und typischen KI-Floskeln.
Summary	Zusammenfassung oder Beschreibung in Plex.
Translate	Workflow zur Übersetzung vorhandener englischer Texte ins Deutsche.
Verification State	Zustand eines geprüften Fakts: verified, probable, weak, conflicting oder unknown.
Wikipedia	Enzyklopädische Quelle für Künstler- und Albumkontext.

14.1 13.1 Empfohlene deutsche Begriffe

Englisch	Im Handbuch bevorzugt
----------	-----------------------

Preview	Vorschau oder Preview
---------	-----------------------

Englisch	Im Handbuch bevorzugt
Review	Review oder Prüfung
Apply	Apply oder Schreiben
Provider	Provider oder Datenquelle
Prompt	Prompt
Summary	Zusammenfassung
Cache	Cache

Einige englische Begriffe bleiben erhalten, weil sie auch in den CLI-Befehlen verwendet werden.

Kapitel 15

14. Anhang

Der Anhang sammelt Befehlsmuster, Dateipfade und praktische Checklisten.

15.1 14.1 Wichtige Befehle

```
plex-enhancer version
plex-enhancer doctor
plex-enhancer login
plex-enhancer scan
plex-enhancer plan --library "Music"
plex-enhancer preview --artist "Jennifer Rush" --album "Credo"
plex-enhancer review --artist "Jennifer Rush" --album "Credo"
plex-enhancer apply --artist "Jennifer Rush" --album "Credo"
plex-enhancer cache stats
```

15.2 14.2 Häufige Exportpfade

Pfad	Inhalt
exports/libraries.json	Bibliotheksstatistik aus <code>scan</code> <code>--export-json</code> .
exports/artists.json	Künstlerexport aus <code>scan artists</code> <code>--export-json</code> .
exports/albums.json	Albumexport aus <code>scan albums</code> <code>--export-json</code> .
exports/audit/	Audit-Datensätze aus Apply- und Analyseabläufen.
exports/backups/	Sicherungen alter Plex-Zusammenfassungen.
exports/context/	Gespeicherte Kontextdokumente.

Pfad	Inhalt
exports/previews/	Gespeicherte Preview-Dokumente.
exports/jobs/	Fortschrittsdaten für Resume-fähige Abläufe.

15.3 14.3 Konfigurationsbeispiele

Minimale Plex-Konfiguration:

```
PLEX_ENHANCER_PLEX_URL=http://localhost:32400
PLEX_ENHANCER_PLEX_TOKEN=IhrPlexToken
```

OpenAI-Konfiguration:

```
PLEX_ENHANCER_AI__PROVIDER=openai
PLEX_ENHANCER_AI__MODEL=gpt-5.5
OPENAI_API_KEY=IhrOpenAIKey
```

15.4 14.4 Sicherer Einzelalbum-Ablauf

1. `plex-enhancer doctor`
2. `plex-enhancer preview --artist "Jennifer Rush" --album "Credo" --verbose`
3. `plex-enhancer review --artist "Jennifer Rush" --album "Credo"`
4. Text prüfen und bei Bedarf bearbeiten.
5. `plex-enhancer apply --artist "Jennifer Rush" --album "Credo"`
6. Audit- und Backup-Dateien aufbewahren.

15.5 14.5 Sicherer Bibliotheksablauf

1. `plex-enhancer library plan --library "Music"`
2. CREATE, TRANSLATE, IMPROVE, REVIEW und SKIP prüfen.
3. Mit kleinem Limit oder Review-Session starten.
4. Regelmäßig Reports erzeugen.
5. Backups nicht löschen, bevor die Ergebnisse geprüft wurden.

15.6 14.6 Exit-Code-Orientierung

Ein erfolgreicher Befehl beendet sich mit Status 0. Fehlerhafte Konfiguration, Verbindungsprobleme oder nicht erfüllte Voraussetzungen führen zu einem Fehlerstatus und einer erklärenden Meldung.

15.7 14.7 Empfehlungen für produktive Nutzung

- Beginnen Sie mit wenigen Alben.
- Nutzen Sie `--verbose`, wenn ein Ergebnis unerwartet wirkt.
- Verwenden Sie `--json` oder `--save`, wenn Ergebnisse archiviert werden sollen.
- Löschen Sie Backups erst nach manueller Prüfung.
- Halten Sie Provider-Schlüssel privat.
- Prüfen Sie große Bibliotheken zuerst mit `plan`, nicht direkt mit `Apply`.

15.8 14.8 PDF-Erstellung

Das PDF-Skript liegt unter:

```
docs/pdf/build.sh
```

Es setzt Pandoc voraus und kombiniert die Kapitel in numerischer Reihenfolge.

Die Ausgabedatei hat immer denselben stabilen Pfad:

```
assets/pdf/Plex-Music-Enhancer-Handbuch.pdf
```

Versionsnummern stehen im Handbuch selbst, aber nicht im Dateinamen.

Das Handbuchlogo wird aus dieser Datei geladen:

```
assets/logo/plex-music-enhancer-logo.pdf
```

Die SVG-Logo-Datei bleibt für GitHub, README und Web-Darstellung erhalten. Der PDF-Build nutzt keine SVG-Konvertierung und benötigt kein Inkscape.

15.9 14.9 Backend-API und REST

Plex Music Enhancer enthält eine interne Backend-API-Schicht unter `plex_music_enhancer.api`, eine optionale FastAPI-REST-Schicht unter `plex_music_enhancer.web` und eine erste React-Weboberfläche im Repository-Verzeichnis `web/`.

Die Schicht definiert:

- versionierte Request- und Response-Modelle für v1
- ein zentrales `ReviewDocument`
- Analysemodelle für Prompt, Qualität, Editorial, Verifikation und Debug-Metadaten
- eine gemeinsame Fehlerhierarchie
- interne Service-Adapter wie `ReviewAPIService` und `ApplyAPIService`
- FastAPI-Router für System, Config, Provider, Logs, Review, Preview und Apply

Der REST-Server wird optional installiert und gestartet:

```
python -m pip install ".[web]"
plex-enhancer serve
```

Wichtige URLs:

URL	Inhalt
http://127.0.0.1:8080/	Weboberfläche
http://127.0.0.1:8080/api/v1/docs	Swagger UI
http://127.0.0.1:8080/api/v1/redoc	ReDoc
http://127.0.0.1:8080/api/v1/openapi.json	OpenAPI JSON
/api/v1/system/health	Health Check
/api/v1/review/artist	Artist Review
/api/v1/review/album	Album Review
/api/v1/preview	Preview
/api/v1/apply	Apply
/api/v1/debug/prompt	letzter Prompt mit Größenanalyse
/api/v1/debug/meta	Prompt-Metadaten
/api/v1/debug/review	strukturierter Review-Log
/api/v1/debug/explain	Explain View
/api/v1/debug/doctor	Developer-Diagnose

CLI, JSON-Ausgabe und Weboberfläche arbeiten auf denselben Backend-Services. Die Web-oberfläche enthält keine Geschäftslogik und konsumiert ausschließlich die REST-API.

Die erste Weboberfläche enthält:

- Dashboard mit Systemstatus und Bibliotheksmetriken
- Künstler- und Albumlisten mit Suche, Filter, Sortierung, Mehrfachauswahl und Kontextaktionen
- Review-Seite mit aktueller Beschreibung, generiertem Text, Monaco Diff und Analyseleiste
- Prompt Debug mit Prompt, Prompt-Metadaten und Review-Log
- Live Log mit Suche und Level-Filter auf vorhandenen Debug-Ausgaben
- Developer-Seite mit Explain View, Prompt Decisions und Diagnosekarten
- REST Explorer für vorhandene API-Endpunkte, Statuscodes und Antwortzeiten
- Einstellungen für Konfiguration und Provider-Status

Der Developer-Mode-Schalter in der Weboberfläche blendet zusätzliche Diagnosebereiche ein. Er löst keine neuen AI-Anfragen aus und verändert keine Schreiblogik.

Ein Klick auf **Review** in einer Liste öffnet die Review-IDE im Hauptbereich und nutzt denselben REST-Endpunkt wie die CLI-nahen Workflows.

Geplante Phasen:

1. Architektur und Contracts
2. interne Backend-API
3. FastAPI
4. REST-Endpunkte
5. React-Oberfläche (erste Version umgesetzt)
6. Desktop-App

15.10 14.10 Developer Mode

Der Developer Mode hilft bei Prompt-Entwicklung, Review-Analyse und Qualitätsdiagnose. Er führt keine AI-Anfragen erneut aus und liest nur bereits vorhandene Debug-Dateien.

Wichtige Dateien:

Datei	Inhalt
/tmp/openai_prompt.txt	letzter an OpenAI gesendeter Prompt
/tmp/openai_prompt_meta.json	Prompt-Metadaten, Budget und Decisions
/tmp/plex_review.log	strukturierter Review-Debug-Log

Befehle:

```
plex-enhancer debug prompt --stats
plex-enhancer debug meta
plex-enhancer debug review --summary
plex-enhancer debug review --section coverage
plex-enhancer debug explain
plex-enhancer debug doctor
```

`debug explain` fasst zusammen, welche Quellen genutzt wurden, welche Evidence entfernt oder gekürzt wurde, wie die Prompt Efficiency aussieht und welche Missed Opportunities im Ergebnis sichtbar sind.

Alle Developer-Mode-Befehle unterstützen `--json`, damit Debugdaten leicht in GitHub-Issues übernommen werden können.

Kapitel 16

15. Container, Portainer und Synology

Plex Music Enhancer kann als vollständig containerisierte Anwendung betrieben werden. Der Container startet die bestehende Weboberfläche über:

```
plex-enhancer serve --host 0.0.0.0
```

Der interne Port ist 8080. Auf Synology kann beispielsweise Host-Port 1008 auf Container-Port 8080 gemappt werden.

16.1 15.1 Docker Image

Das Image wird über GitHub Container Registry veröffentlicht:

```
ghcr.io/amlasehai0z/plex-music-enhancer
```

GitHub Actions erzeugt Tags für `latest`, `main`, optional `develop` und Git-Versionstags.

Veröffentlichte Images werden für diese Plattformen gebaut:

- `linux/amd64`
- `linux/arm64`

Damit läuft dasselbe Image auf Intel-/AMD-Systemen und auf ARM-basierten Synology-Systemen.

Wichtige Tags:

Tag	Zweck
<code>latest</code>	aktuelles Image des Standard-Branches
<code>main</code>	aktuelles Image des <code>main</code> -Branches
<code>develop</code>	optionales Integrations-Image
<code>v1.0.0</code>	unveränderliches Release-Image

16.2 15.2 Docker Compose

Das Repository enthält eine produktionsnahe `docker-compose.yml`. Die Datei kann ohne Anpassungen als Portainer Stack importiert werden.

Wichtige Einstellungen:

Einstellung	Wert
Containername	plex-music-enhancer
Restart Policy	unless-stopped
Portmapping	1008:8080
Healthcheck	/api/v1/system/health

Start:

```
docker compose up -d
```

Lokale Docker-Validierung:

```
docker build -t plex-music-enhancer:local .
docker compose config
docker run --rm plex-music-enhancer:local plex-enhancer --help
docker run --rm plex-music-enhancer:local plex-enhancer serve --help
docker compose up -d
until curl --fail --silent http://127.0.0.1:1008/api/v1/system/health; do sleep 1; done
docker compose down
```

Der Healthcheck verwendet einen bestehenden REST-Endpunkt. Es ist keine zusätzliche API erforderlich.

16.3 15.3 Volumes

Containerpfad	Zweck
/config	Konfiguration und optionale <code>.env</code>
/cache	Provider- und Knowledge-Cache
/logs	Prompt- und Review-Debugdateien
/music	optionaler read-only Musikpfad

16.4 15.4 Umgebungsvariablen

Variable	Zweck
OPENAI_API_KEY	OpenAI API-Key
PLEX_URL	Kurzform für Plex URL
PLEX_TOKEN	Kurzform für Plex Token
PLEX_ENHANCER_PLEX_URL	Plex URL
PLEX_ENHANCER_PLEX_TOKEN	Plex Token
PLEX_ENHANCER_CONFIG	Konfigurationsordner oder <code>.env</code> -Datei, Standard <code>/config</code>
PLEX_ENHANCER_CACHE	Cachepfad, Standard <code>/cache</code>
PLEX_ENHANCER_LOG_LEVEL	Log-Level
PLEX_ENHANCER_WEB__PORT	interner Webport, Standard 8080

Secrets gehören nicht ins Image. Verwenden Sie Container-Environment oder `/config/.env`.

16.5 15.5 Portainer

Portainer ist die empfohlene Verwaltungsoberfläche für den Docker-Betrieb von Plex Music Enhancer. Aktualisierungen erfolgen bewusst manuell: Sie entscheiden, wann ein neues Image aus GHCR übernommen und der Container neu gestartet wird. Automatische Container-Updates werden nicht empfohlen.

Der offizielle Ablauf lautet:

```

Git Push
↓
GitHub Actions
↓
Tests
↓
Docker Image Build
↓
Push nach GHCR
↓
Portainer
↓
Pull des neuen Images
↓
Container neu starten

```

16.5.1 Deployment als Stack

1. Portainer öffnen.
2. **Stacks** auswählen.
3. Einen Stack mit dem Namen `plex-music-enhancer` anlegen.
4. Die `docker-compose.yml` aus dem Repository einfügen oder hochladen.
5. Environment-Variablen in Portainer setzen oder eine Env-Datei verwenden.
6. Stack deployen.
7. Weboberfläche über `http://<host>:1008/` öffnen.

16.5.2 Deployment als einzelner Container

1. In Portainer **Containers** öffnen.
2. Image `ghcr.io/amylasehai0z/plex-music-enhancer:latest` auswählen.
3. Container-Port 8080 setzen.
4. Host-Port 1008 auf Container-Port 8080 mappen.
5. Volumes für `/config`, `/cache` und `/logs` einhängen.
6. Environment-Variablen setzen.
7. Restart Policy `unless-stopped` verwenden.
8. Container starten und Healthcheck prüfen.

16.5.3 GHCR-Image

Für aktuelle Releases verwenden Sie:

```
ghcr.io/amylasehai0z/plex-music-enhancer:latest
```

Für Rollbacks kann stattdessen ein älterer Release-Tag verwendet werden, zum Beispiel:

```
ghcr.io/amylasehai0z/plex-music-enhancer:v1.0.1
```

16.5.4 Volumes

Containerpfad	Zweck
<code>/config</code>	Konfiguration und optionale <code>.env</code>
<code>/cache</code>	Provider- und Knowledge-Cache
<code>/logs</code>	Prompt- und Review-Debugdateien
<code>/music</code>	optionaler read-only Musikpfad

16.5.5 Environment-Variablen

```
OPENAI_API_KEY=
PLEX_URL=http://plex:32400
PLEX_TOKEN=
PLEX_ENHANCER_WEB__PORT=8080
PLEX_ENHANCER_LOG_LEVEL=INFO
PLEX_ENHANCER_CONFIG=/config
PLEX_ENHANCER_CACHE=/cache
```

16.5.6 Port-Mapping und Healthcheck

Empfohlenes Port-Mapping:

```
1008:8080
```

Der Healthcheck verwendet:

```
http://127.0.0.1:8080/api/v1/system/health
```

16.5.7 Container aktualisieren

1. Stack oder Container in Portainer öffnen.
2. Neues Image aus GHCR pullen.
3. Container neu erstellen oder neu starten.
4. Healthcheck prüfen.
5. Weboberfläche öffnen und Version kontrollieren.

16.5.8 Rollback, Logs und Neustart

Für ein Rollback wird in Portainer ein älterer GHCR-Tag gesetzt und der Stack oder Container erneut deployt. Logs sind in Portainer direkt auf der Container-Detailseite sichtbar. Der Container kann dort auch manuell neu gestartet werden.

16.6 15.6 Synology Container Manager

1. Container Manager öffnen.
2. Image `ghcr.io/amylasehai0z/plex-music-enhancer:latest` verwenden oder das Compose-Projekt importieren.
3. Port 1008 auf 8080 mappen.
4. `/config`, `/cache` und `/logs` als persistente Ordner einhängen.
5. `PLEX_URL`, `PLEX_TOKEN` und `OPENAI_API_KEY` setzen.
6. Neustartregel `unless-stopped` verwenden.
7. Healthcheck prüfen.
8. Weboberfläche öffnen: `http://<synology>:1008/`.

16.7 15.7 CI/CD

Der GitHub Actions Workflow führt Tests, Linting, Frontend-Build, Paket-Build, Docker-Smoke-Build, Container-Smoke-Tests, Health-Endpunkt-Smoke-Test, Multi-Arch-Build, SBOM, Provenance und GHCR-Veröffentlichung aus.

Veröffentlicht werden Container-Images nur für:

- `main`
- `develop`
- Release-Tags wie `v1.0.0`

Pull Requests und Feature-Banches validieren ohne Image-Veröffentlichung. So bleiben Tests zuverlässig, während Portainer nur bewusst freigegebene Images übernimmt.

Kapitel 17

16. Release Management

Plex Music Enhancer verwendet einen tag-basierten Release-Prozess. Ein Versions-Tag reicht aus, um Tests, Paketbau, Container-Build, GHCR-Publish und GitHub Release auszulösen.

17.1 16.1 Empfohlener Release-Prozess

```
git tag v1.0.0
git push origin v1.0.0
```

Danach läuft automatisch:

```
Git Tag
↓
GitHub Actions
↓
Tests
↓
Python Wheel
↓
Source Distribution
↓
Docker Smoke Tests
↓
Multi-Arch Docker Image
↓
GHCR
↓
GitHub Release
↓
Release-Artefakte
```

17.2 16.2 GitHub Releases

Für Tags wie `v1.0.0` erstellt oder aktualisiert GitHub Actions automatisch ein GitHub Release. Der Release-Titel folgt dem Schema:

```
Plex Music Enhancer v1.0.0
```

Release Notes werden aus GitHub-Daten erzeugt und bei erneutem Workflow-Lauf aktualisiert.

17.3 16.3 Release-Artefakte

Jedes Release erhält automatisch:

- Python Wheel (`.whl`)
- Source Distribution (`.tar.gz`)
- Build Report
- Docker Analyse
- Release Readiness Report

Diese Dateien werden zusätzlich als GitHub Actions Artifacts gespeichert.

17.4 16.4 GHCR und Image Tags

Release-Tags erzeugen passende Images in GitHub Container Registry.

Beispiele:

Tag	Zweck
<code>latest</code>	aktuelles Image des Standard-Branche
<code>main</code>	aktuelles Image des <code>main</code> -Branche
<code>v1.0.0</code>	unveränderliches Release-Image
<code>1.0</code>	SemVer-Minor-Tag

Die Images werden für `linux/amd64` und `linux/arm64` gebaut. Dadurch sind Intel-/AMD-Systeme und ARM-basierte Synology-Systeme abgedeckt.

17.5 16.5 Build Report

Der Build Report dokumentiert technische Eckdaten:

- Git Commit
- Branch oder Tag

- Version
- Python Version
- Docker Version
- Docker Compose Version
- Image ID
- Imagegröße
- Wheel- und Source-Distribution-Größe
- OCI Labels
- Multi-Arch, SBOM und Provenance
- Healthcheck
- Smoke-Test-Ergebnis
- Docker-Compose-Test
- Build-Dauer
- Zeitstempel

17.6 16.6 Docker Analyse

Die Docker Analyse ergänzt:

- Imagegröße
- Anzahl Layer
- größte Layer
- Cache-Konfiguration
- mögliche Optimierungspotentiale

Sie ist rein diagnostisch und verändert das Image nicht.

17.7 16.7 Release Readiness Report

Der Release Readiness Report bewertet:

- Packaging
- Docker
- Docker Compose
- GitHub Actions
- GHCR
- Portainer
- Deployment
- Dokumentation

Die Bewertung ist bewusst technisch und soll verbleibende Risiken sichtbar machen, statt künstlich perfekte Ergebnisse zu melden.

17.8 16.8 Portainer nach dem Release

Nach einem Release zieht der Anwender das neue GHCR-Image in Portainer manuell und startet den Container neu. Automatische Container-Updates werden nicht empfohlen. Für Rollbacks wird ein älterer Release-Tag ausgewählt.

Kapitel 18

Anhang zur PDF-Ausgabe

18.1 Versionshistorie

Version	Schwerpunkt
1.0	Erstes vollständiges deutsches Benutzerhandbuch.
1.1	Professionelles PDF-Publishing mit XeLaTeX, Layoutvorlage und Build-Skripten.

18.2 CLI-Referenz

Die vollständige Befehlsübersicht steht in Kapitel 5. Für die aktuell installierte Version kann zusätzlich jederzeit die eingebaute Hilfe verwendet werden:

```
plex-enhancer --help
plex-enhancer preview --help
plex-enhancer library --help
```

18.3 Lizenz

Plex Music Enhancer steht unter der MIT License. Der vollständige Lizenztext befindet sich in der Datei LICENSE im Projektverzeichnis.